

PYTHON

Module 4: Logical Statements

CODING

Defining Python Statements

At its core, a computer program is a sequence of 'instructions' that the computer processes. In the world of programming, these specific instructions are called statements.

A statement is the smallest standalone element of a programming language that expresses some action to be carried out.

Example:

```
print('Statement execution starts here.')
```

In this case, the `print()` function is a statement that tells the computer to output text to the console.

Sequential Execution Flow

Python follows a 'Top-to-Bottom' execution model. This means that statements are processed one by one, in the exact order they are written in the source file.

Workflow Example:

1. Statement A: ``x = 5`` (Assignment)
2. Statement B: ``y = 10`` (Assignment)
3. Statement C: ``print(x + y)`` (Output)

Even in complex applications, this linear progression forms the backbone of the program's logic.

Ending a Statement: Newlines

In languages like C++ or Java, every statement must end with a semicolon (;). In Python, the design is much cleaner.

Python identifies the end of a statement by the end of the line (newline). This removes unnecessary characters and makes the code resemble natural English.

Rule: One line, one instruction. This simplicity is one of the primary reasons Python is favored for rapid software development.

The Semicolon Exception

While end-of-line newline is the standard, Python does technically allow semicolons (;) to separate multiple statements on a single line.

Technique:

```
print('Hi'); x = 5; print(x)
```

CRITICAL NOTE: While functional, this is considered 'un-pythonic' and bad practice. It reduces readability and makes debugging difficult. The CodeHive standard recommends placing every statement on its own dedicated line.

Compound Statements

Beyond simple single-line instructions, Python uses 'Compound Statements.' These allow you to group multiple statements together to form complex logic.

Common Compound Structures:

- ``if`` statements: Conditional execution.
- ``for`` & ``while`` loops: Repeated execution.
- ``def``: Function definitions.

These rely on the colons (`:`) and indentation patterns we covered in the Syntax module.

Syntax Errors in Statements

A program will fail to run if Python cannot interpret a statement correctly. The most common error is attempting to put two statements together without a separator.

Error Scenario:

```
print('Hello') print('World') – This will trigger a  
SyntaxError: invalid syntax .
```

Fix: Place the second print on a new line or use a semicolon as a temporary (though discouraged) bridge.

Summary & Best Practices

To write high-quality Python code at CodeHive, follow these statement-writing rules:

- **Clarity:** Prioritize a clean vertical flow of logic.
- **Readability:** Avoid 'crowding' your lines with multiple instructions.
- **Structure:** Use blank lines between groups of related statements to improve visual hierarchy.

Programmers spend more time reading code than writing it—always write for the human reader!